

→ Object oriented programming in python.

OOPs

Till now procedural python (one line after other)

Classes, objects



Blueprint/Layout

irrespective of object

all objects have the features of class that it is created in.

Objects are the instances of a particular class.

```
class person():  
    pass
```

```
john-doe = Person()  
john-doe.name = 'Alec'  
john-doe.surname = 'Baldwin'  
john-doe.dob = 1958
```

class person: ^{initialization / fixing of the template.}

def __init__(self, name, surname, year-of-birth)

self.name = name.

~~self~~ self.surname = surname

self.year-of-birth = year-of-birth.

self is object ??

↳ is the instance object name.

first variable in __init__ is always the instance reference.

1.2 Methods.

↳ all the instances created in the class have all the methods in the class associated with instance.

print(object)

↳ gives the reference memory of the object.

def __str__(instance)

↓
now print(object)

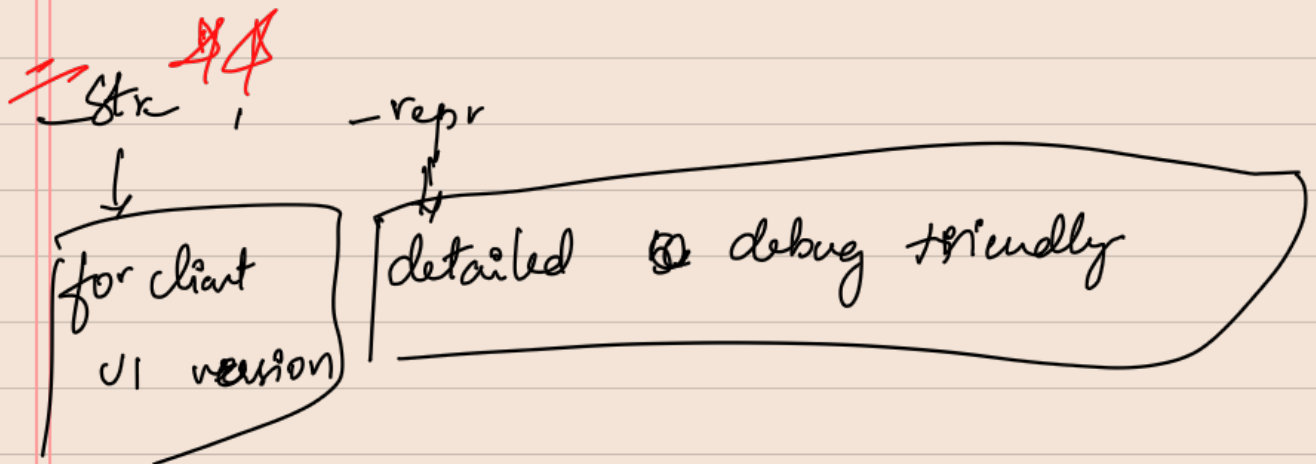
↳ will print the actual content instead of reference.

→ Bad practice.

class Person

setters can also be used instead of
__init__

but this is bad practice.



1.4 abstraction :-

Hides the internal implementation. while exposing only the ne

no private/public in python.

- is used for abstraction.

-- is the main object protection.

- Person -- surname. is used for abstraction

→ getters are used to encapsulate the actual methods and object variables finally giving us the abstraction.

↓
protecting sensitive information.

→ --dict--

↓
represent the object instance as a dictionary

→ Inheritance:-

Parent properties are inherited to child and child has its own properties.

Subclass → super class.
Class Student (Person) :-
def __init__(self, student_id, *args) ^{inherited props.}
super(Student, self).__init__(*args)
self.__student_id__ = student_id.

~~or~~ Kwargs → Key word arguments

def demo(a, b, c, *args, **Kwargs).

positional arguments

↓
we will pass the parameters in correct order of super class

demo (1, 2, 3, 4, 5, 7, n=10, y=20)

↓ ↓ ↓ ↓ Kwargs
a y c args

positional argument follows keyword arguments

2.1 overriding Method:-

only modifies the method at child level.

changing the behaviour of method.